Symmetry-filtered read-out on IBM quantum devices: the qg filter on twenty device noise models through IBM Runtime

Vicente Humberto Monteverde
Universidad del Museo Social Argentino (UMSA), Argentina
ORCID 0000-0001-8884-4811

Draft of 13 October 2026

Abstract

Circuits built from gates that conserve the Hamming weight keep their state in one weight sector, so the shots that left the sector can be discarded at read-out: the qg filter of the open-source library qang [1], a form of symmetry verification [2, 3]. We make it usable on IBM devices through IBM Runtime and Qiskit, and measure it on the noise models of twenty IBM devices. The library adds a Qiskit transpiler pass that decides whether the filter applies, read-out correction from each device's published per-qubit errors, qg read-outs of IBM Runtime Sampler and Estimator results, zero-noise extrapolation by gate folding that stacks with the filter, a prediction of the filter's kept fraction from the calibration, qubit selection with that prediction, a shot planner, and the weight-conserving quantum neural network (QNN) of qang as one parameterized job and as a qiskit-machine-learning `SamplerQNN`. Every prediction was committed before its run and is reported as passed or failed. On ten five-qubit weight-1 circuits the filter lowers the error of the measured $qg_Z = \langle Z \rangle$ from 0.065–0.104 to 0.018–0.023 on three devices (0.013–0.022 with read-out correction), and filter plus zero-noise extrapolation reaches 0.011–0.015. On seventeen further devices never used before, the filter cuts the error 2.4–8.9× on every one (median 3.1× with read-out correction). Choosing the qubits by the predicted kept fraction keeps 6–17 points more shots than Qiskit's default layout on chain circuits, avoids a qubit published as reading 1 always, and the shot planner met a 0.02 target precision on three devices. For classification the gain is small: the QNN recovers 5–6 of the 10 decisions noise costs it. Two interoperability bugs were found, one in qiskit-machine-learning and one in the library. All results are on device noise models; no hardware run is reported.

Keywords: quantum error mitigation; symmetry verification; post-selection; IBM Quantum; quantum machine learning.

1 Introduction

When every gate of a circuit conserves the number of excitations (the Hamming weight of the computational state), noise that changes that number moves shots out of the sector the circuit started in. Discarding them is symmetry verification by post-selection [2, 3]. In qang it is the qg filter, read in the unit $qg_Z = \langle Z \rangle \in [-1, 1]$ [1]; a companion note studies it for quantum neural networks under relaxation [4].

This paper asks what the filter is worth on IBM devices and gives the tools to use it there: everything runs through IBM Runtime [5] and Qiskit [6], here in local testing mode on the device noise models (fake backends), unchanged on a device. Each experiment fixed its circuits, settings and predictions in a commit before running; failures are reported as such. Sections §137–§144 refer to the research notes of the repository, where every number below is reproduced by a script in `examples/`.

2 Methods

Applicability. `WeightPreservationCheck` is a Qiskit analysis pass. It marks a circuit as weight-preserving when every gate conserves the excitation number (by name for gates such as RZ, CZ, SWAP or XX+YY, otherwise from its matrix), and returns the initial weight prepared by leading X gates. It is run on the logical circuit: after transpilation the native gates are not weight-preserving one by one.

Read-out. For a circuit transpiled to a device, `ibm_qg` reads the measured physical qubits, takes their published asymmetric read-out errors $P(1|0)$, $P(0|1)$, and inverts the tensor product of the per-qubit confusion matrices on the measured distribution [7]. It returns qg_Z per qubit raw, read-out corrected, filtered (weight-w shots of the raw distribution) and filtered after correction (the weight-w part of the corrected quasi-distribution, renormalized). Qubits whose confusion matrix cannot be inverted are left uncorrected and reported.

Kept fraction and qubit selection. `predict_kept_fraction` multiplies the probability that each calibrated channel leaves the weight unchanged: a single-qubit gate of error e changes it with probability e, a two-qubit gate with $5e/6$, relaxation over the busy time t of a qubit with $(1 - e^{-t/T_1})\, w/n$, pure dephasing during the gates with $2z/3$, $z = (1 - e^{-t/T_\phi})/2$, and read-out with $\frac{w}{n}P(0|1) + (1 - \frac{w}{n})P(1|0)$. Adding the shots that return to the sector as for a fully mixed state gives `kept_mixed_floor`. `best_chain` and `best_layout` choose the initial layout with the highest prediction; `best_layout` scores the circuit as it will run, routing included, and counts Qiskit's default layout among the candidates.

Zero-noise extrapolation. IBM Runtime's own mitigation has no effect in local mode, so `zne_qg` folds every two-qubit gate ($G \to GG^\dagger G$, scale 3) [8], runs both scales and extrapolates linearly to zero [9]. Folding keeps a weight-preserving circuit weight-preserving, so it stacks with the filter.

Shot planning. A filtered qg_Z read from Nk kept shots has standard error $\sqrt{(1 - qg_Z^2)/(Nk)}$. `plan_shots` returns the N that brings the worst qubit to a target error with the predicted k, and the device time N(circuit duration + repetition delay).

QNN. The weight-1 QNN of qang [4] (an amplitude loader and three layers of Hamming-weight-preserving RBS gates on five qubits, ring-coupled) is written as one parameterized circuit of native XX+YY gates, $\mathrm{RBS}(t) = \mathrm{XXPlusYY}(2t, \pi/2)$, equal to the library's simulator to 10^{-9}. `run_qnn` sends a whole test set as one Sampler pub. `qang_sampler_qnn` wraps the same circuit as a qiskit-machine-learning `SamplerQNN` [10]. The model is trained noiselessly on four datasets (iris, cancer, wine, digits; 189 test inputs; 185 correct noiselessly).

Test circuits. Ten random five-qubit weight-1 circuits: X on qubit 0 and two layers of XX+YY gates on neighbours with angles uniform in $[0.3, 2.8]$ (seeds 0–9). Error is the mean $|qg_Z - qg_Z^{\mathrm{ideal}}|$ over qubits and circuits. Transpilation uses optimization level 1 and seed 137.

3 Results

3.1 Read-out on three devices (§137, §142)

All three predictions of §137 passed: read-out correction lowers the raw error (by 29–64%), the filter does more than read-out correction alone, and the two together are best (Table 1). Read-out correction raises the weight-1 mass from 0.69–0.82 to 0.82–0.92: a third to two thirds of the

	raw	read-out	filter	filter + read-out	ZNE	filter + ZNE
FakeBrisbane	0.104	0.074	0.023	0.022	0.053	0.012
FakeSherbrooke	0.065	0.038	0.019	0.016	0.034	0.011
FakeTorino	0.102	0.036	0.018	0.013	0.085	0.015

Table 1: Mean qg_Z error, ten circuits; 8000 shots per circuit (and per noise scale for ZNE). The first four columns are §137, the ZNE columns §142 (separate runs).

shots the filter would discard were read-out errors. With ZNE (§142) the filter does 2–7× more than ZNE alone (passed); ZNE lowered the filtered and corrected error on two devices but not on Torino, and the best combination was filter + ZNE on two devices and filter + read-out on Torino (so both of those predictions failed).

3.2 Twenty devices (§144)

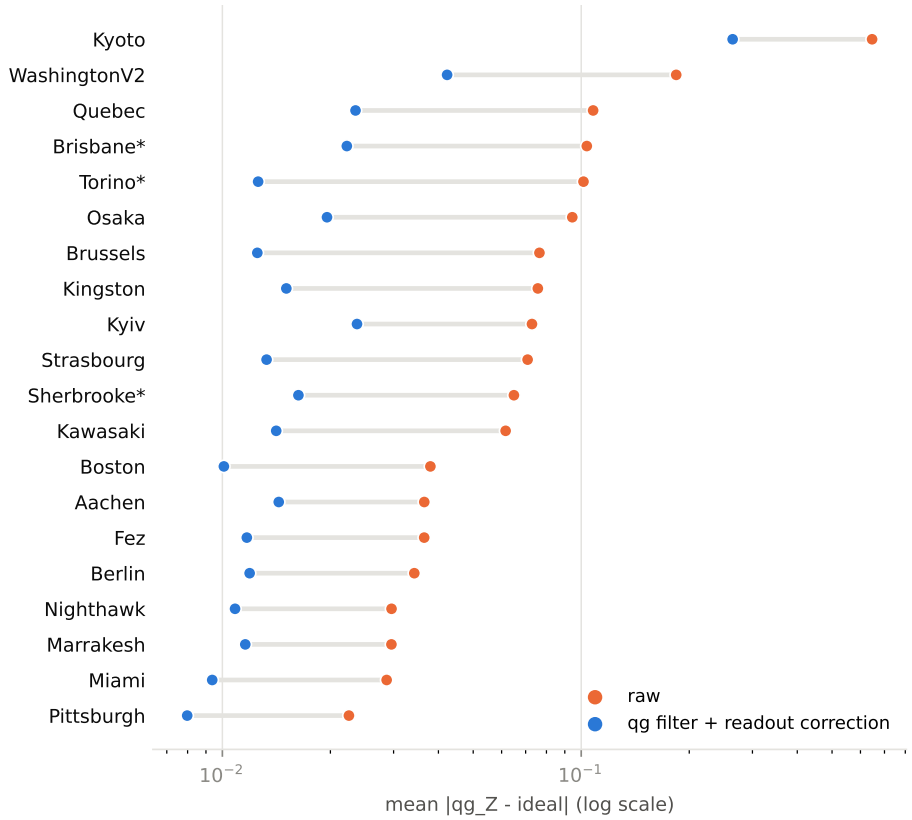


Figure 1: Raw and filtered (+ read-out corrected) qg_Z error on twenty IBM device noise models. Starred: §137 (ten circuits, 8000 shots); the others: §144 (five circuits, 4000 shots), devices not used before.

On the eighteen other fake backends with at least 100 qubits (ECR, CZ and CX devices of 120–156 qubits), the filter cut the error 2.4–8.9× on every one of the seventeen that ran (Fig. 1; median raw / filtered + read-out 3.12). The three predictions nonetheless failed as written: one device crashed (two predictions required all eighteen), and filter + read-out was lowest on 10 of 17, the filter alone on the other 7, where read-out errors are small and correcting them adds variance. The crash was a bug of the library: FakeCusco publishes a qubit that always reads 1, the transpiler placed the circuit on it, and the read-out inverse was singular. After the fix,

`best_layout` avoids that qubit and brings the filtered error from 0.098 to 0.021 (exploratory).

3.3 Predicting the kept fraction and choosing qubits (§138, §141)

Over ten layouts per device the prediction ranked the layouts correctly (Spearman 0.94–0.98, passed) and `best_chain` kept 6–17 points more shots than Qiskit's default (passed), but the raw prediction was low by 0.08–0.13 (failed): it ignored shots that return to the sector. With the mixed-state floor, added after scoring, the error is 0.023–0.041. For the ring-coupled QNN, `best_chain` scored a different circuit from the one that ran and kept fewer shots on two devices (§139); `best_layout` fixes this: it was never worse than the default and gained 2.2 points where it chose a different layout (§141, two of three predictions passed).

3.4 Shot planning (§143)

For a target error of 0.02 the planner asked for 3000–3400 shots (0.8–0.9 device seconds); over 30 repetitions the worst qubit's error was 0.021–0.023 (passed). The per-qubit prediction was 1.4–1.6× too low on average (failed): qubits near a pole ($|qg_Z| > 0.8$) are underestimated 2.4–3.1×, because noise pulls their qg_Z away from ± 1; the plan is set by the worst qubit and still met the target.

3.5 The QNN (§139–§141)

	raw	filter	filter + read-out
`run_qnn`, default layout (§139)	545	550	549
`run_qnn`, `best_chain` (§139)	547	551	551
`run_qnn`, `best_layout` (§141)	545	549	550
qiskit-machine-learning `SamplerQNN` (§140)	547	548	–

Table 2: Correct decisions of 567 (189 inputs on each of three devices; 555 noiselessly), 1000 shots.

The one-job template agrees with per-input circuits (550 against 551, passed) and qiskit-machine-learning reproduces the library's own path (548, passed), but the filter recovers only 5–6 of the 10 decisions noise costs, and read-out correction changes no decision (Table 2). Runs with the identical layout differ by up to two decisions, so differences of that size are shot noise. Wrapping the QNN in qiskit-machine-learning 0.9.1 exposed that, with a device pass manager, `SamplerQNN` reads measured bits at the physical-qubit positions of the layout, so a circuit measured before transpilation reads every shot as 0 unless its layout is $[0, \ldots, n-1]$; the library transpiles itself and drops the layout, and the issue was reported upstream.

4 Discussion

As a read-out tool the filter is robust: on every one of twenty IBM device noise models it lowered the qg_Z error by a factor of 2.4 or more, and it combines with read-out correction and with zero-noise extrapolation. Which combination is best depends on the device: read-out correction helps where read-out errors are large and adds variance where they are small. The calibration also tells, before any device time is spent, how many shots the filter will keep, which qubits keep most, and how many shots a target precision needs.

The limits are as clear. The gain in classification accuracy is small, because the model's decisions are mostly far from threshold. Nine of the twenty-three predictions failed as written, several through the analysis rather than the filter: a prediction that ignored returning shots, a per-qubit error model that ignored the poles, and an experiment that required every device

to run. And all results are on device noise models, which model gate, relaxation, dephasing and read-out errors as independent channels; leakage, crosstalk and drift are absent. A run on hardware is the next step.

Code and data availability

Everything is in the open-source library qang (MIT license), version 0.9.0 and later: https://github.com/Viny2030/qang, `pip install qang`. The module is `qang.ibm` (with `qang.qiskit_ml`); each section has a script in `examples/` and its output in `examples/data/`; `docs/ibm_quickstart.md` shows the workflow.

```
from qang import ibm
t = transpile(qc, backend) # qc: weight-preserving, measured
res = Sampler(mode=backend).run([t]).result()[0]
ibm.ibm_qg(res, t, backend, weight=1)["filtered_readout"]
```

References

[1] Vicente Humberto Monteverde. The Qang (qg): A unified angular-probability unit and metric for parametric quantum circuit design. 2026. doi: 10.5281/zenodo.22832150.

[2] Xavier Bonet-Monroig, Ramiro Sagastizabal, M. Singh, and Thomas E. O'Brien. Low-cost error mitigation by symmetry verification. *Physical Review A*, 98:062339, 2018.

[3] Sam McArdle, Xiao Yuan, and Simon Benjamin. Error-mitigated digital quantum simulation. *Physical Review Letters*, 122:180501, 2019.

[4] Vicente Humberto Monteverde. Training quantum neural networks for T1-limited hardware: what the qg symmetry filter does and does not do, 2026. Companion note, `manuscript/qml/` in the repository.

[5] IBM Quantum. Qiskit IBM runtime client, version 0.50. https://github.com/Qiskit/qiskit-ibm-runtime, 2026.

[6] Ali Javadi-Abhari et al. Quantum computing with Qiskit, 2024.

[7] Sergey Bravyi, Sarah Sheldon, Abhinav Kandala, David C. Mckay, and Jay M. Gambetta. Mitigating measurement errors in multiqubit experiments. *Physical Review A*, 103:042605, 2021. doi: 10.1103/PhysRevA.103.042605.

[8] Tudor Giurgica-Tiron, Yousef Hindy, Ryan LaRose, Andrea Mari, and William J. Zeng. Digital zero noise extrapolation for quantum error mitigation. In *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*, pages 306–316, 2020. doi: 10.1109/QCE49297.2020.00045.

[9] Kristan Temme, Sergey Bravyi, and Jay M. Gambetta. Error mitigation for short-depth quantum circuits. *Physical Review Letters*, 119(18):180509, 2017. doi: 10.1103/PhysRevLett.119.180509.

[10] Qiskit community. Qiskit machine learning, version 0.9.1. https://github.com/qiskit-community/qiskit-machine-learning, 2026.